

claude-windows / 9f6357d8-0e35-491a-a830-052e094c7f37

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\9f6357d8-0e35-491a-a830-052e094c7f37\subagents\workflows\wf_e3cc36d8-eca\agent-a92eb8537bc1cdd5f.jsonl`  
- SHA-256: `f5da9b40b7e14896a9a0d1993f55cfc1d3871da55d105fcdcfadad3450ca5bb90`  
- Source modified: `2026-06-21T09:25:39+00:00`  
- Imported at: `2026-07-05T16:48:25+00:00`  
- project: `wf_e3cc36d8-eca`  
- session_id: `9f6357d8-0e35-491a-a830-052e094c7f37`
```

Transcript

1. user (2026-06-21T09:22:05.529Z)

Engine repo C:/Users/avidu/Projects/badminton-highlight-indexer; SPA
C:/Users/avidu/Projects/khelsutra/web. Synthesize the final plan for mission "make the bucket?reel CUJ fully UI-driven (Path B)", first item B-P1.

INPUTS:

TRACE (does B-P1 already work in code?): {"file":"backend/storage/ref.py","line":"60-61","what":"parse_uri normalizes scheme 'gs' -> 'gcs' ('if scheme == 'gs': scheme = 'gcs')", so gs://bucket/key becomes a gcs StorageRef(backend='gcs')."}, {"file":"backend/storage/ref.py","line":"87-88","what":"resolve_input_uri: explicit scheme wins first ('if _SCHEME_RE.match(s): return s') ? a gs:// / gcs:// input passes through unchanged regardless of input_backend/input_root, so the DEFAULT local config still honors an explicit bucket URI."}, {"file":"backend/config/models.py","line":"315-316","what":"StorageConfig defaults: input_backend Literal default 'local', input_root default ''. Under DEFAULT config a gs:// path is taken as an explicit-scheme bucket ref (precedence rule 1), not forced local."}, {"file":"backend/config/models.py","line":"283","what":"SecurityConfig.allowed_video_root default None ? single-user mode; validate_input_path only runs the null-byte check."}, {"file":"backend/utlis/paths.py","line":"53-63","what":"validate_input_path(path, allowed_root=None): with allowed_root falsy it ONLY rejects null bytes; it does NOT require a local/existing path, so a gs:// string passes the submit-time validation."}, {"file":"backend/main.py","line":"269-276","what":"POST /api/process: allowed_root=getattr(...security...allowed_video_root, None) (None by default) -> validate_input_path passes; storage.resolve_input_ref(req.video_path, storage_cfg) classifies the gs:// input."}, {"file":"backend/main.py","line":"283-284","what":"Submit-time existence fast-fail is GATED on local: 'if input_ref.backend == 'local' and not os.path.exists(input_ref.key)`. A gs:// input resolves to backend 'gcs', so the os.path.exists 404 check is SKIPPED ? existence is deferred to the worker fetch. No code gate blocks a bucket URI at submit."}, {"file":"backend/main.py","line":"292-296","what":"create_job(job_id, req.video_path, ...) persists the RAW req.video_path (the gs:// string, unmodified) into the jobs table; then _get_executor().submit(_drain_due_jobs) kicks the worker. Endpoint returns 202 + job_id."}, {"file":"backend/infrastructure/database.py","line":"486-499","what":"create_job INSERTS video_path verbatim (no normalization) into jobs(video_path)."}, {"file":"backend/infrastructure/database.py","line":"588-608","what":"claim_due_job flips a queued job to running and returns get_job(id), which does SELECT * ? so the stored gs:// video_path round-trips back to the worker intact."}, {"file":"backend/api/job_worker.py","line":"59-69","what":"_job_to_request rebuilds ProcessRequest(video_path=job['video_path'], ...) from the stored row ? the gs:// string is faithfully reconstructed."}, {"file":"backend/api/job_worker.py","line":"84-90","what":"_run_claimed_job calls _execute_processing(req); on success calls set_job_video_id(job_id, result['video_id']) and mark_job_done ? the computed video_id is recorded on the job for the poller."}, {"file":"backend/main.py","line":"159-161","what":"_execute_processing: local_video = storage.acquire_local_input(req.video_path, global_config.storage) FETCHES the gs:// file to scratch, then video_id = compute_video_id(local_video) (MD5 of the fetched bytes)."}, {"file":"backend/storage/__init__.py","line":"59-74","what":"acquire_local_input: for a non-local ref (gcs), mkdtemp scratch + fetch(ref.uri, scratch/name) -> downloads and returns

the LOCAL path; local refs are identity

```

passthrough."}, {"file": "backend/storage/__init__.py", "line": "106-111", "what": "fetch ->
get_storage('gcs').fetch_to_local(ref, dst) for a gcs
URI."}, {"file": "backend/storage/gcs.py", "line": "30-39", "what": "GCSStorage.fetch_to_local:
makedirs + self._blob(ref).download_to_filename(dst) ? the actual GCS download (lazy `from
google.cloud import storage`)."}, {"file": "backend/main.py", "line": "190", "what": "DB write is
UNCONDITIONAL on the fetched local file: after validators pass, db_instance.add_video(video_id,
req.video_path, 'processed', ...) writes the videos row (filepath = the ORIGINAL gs:// URI,
status
processed)."}, {"file": "backend/infrastructure/database.py", "line": "287-304", "what": "add_video is
an UPSERT (INSERT ... ON CONFLICT(id) DO UPDATE) ? unconditional write/update of the videos row
keyed by
video_id."}, {"file": "backend/main.py", "line": "212-237", "what": "segmenter.process_video(video_id,
local_video, ...) runs on the fetched local path; _finalize_reingest commits the produced play
segments (destroy-after-
confirm)."}, {"file": "backend/pipeline/segmenters/motion.py", "line": "201", "what": "Segmenter
persists each rally via self.db.add_play_segment(video_id, start, end, ...) -> INSERT INTO
play_segments (database.py:341-350). Same write path for gemini.py:492, yolo_hybrid.py:346,
trajectory_hybrid.py:367 ? the play_segments rows are keyed by the MD5
video_id."}, {"file": "backend/main.py", "line": "360-369", "what": "POST /api/query ->
db_instance.query_play_segments(video_id, filters) returns the rows; combined with GET
/api/jobs/{id} exposing video_id, the SPA can load the result via ?video=<video_id>. Round-trip
closed."}], "gaps": [], "runtimeDeps": ["google-cloud-storage SDK installed (the `storage-gcs`
extra) ? imported lazily in backend/storage/gcs.py:24 (`from google.cloud import storage`);
absent => ImportError fails the JOB (loud), not the submit.", "Application Default Credentials
with read access to the bucket (GOOGLE_APPLICATION_CREDENTIALS service-account JSON or workload
identity) ? GCSStorage.__init__ does storage.Client(project=...) with no config-supplied secrets
(gcs.py:18-25).", "Network egress to storage.googleapis.com for download_to_filename
(gcs.py:38).", "ffmpeg on PATH for the validators/segmenter (project-wide pipeline prerequisite,
unrelated to gs://)."], "notes": "CLAIM HOLDS as code. Under DEFAULT single-user config
(security.allowed_video_root=None, storage.input_backend='local', input_root=''), POST
/api/process accepts a gs:// or gcs:// video_path end-to-end with NO code gate blocking
it:\\n\\n1. SUBMIT (main.py:269-300): validate_input_path with allowed_root=None only checks
null bytes (paths.py:60-61) ? a gs:// string passes. resolve_input_ref classifies it via
resolve_input_uri (explicit-scheme-wins, ref.py:87-88) and parse_uri (gs->gcs normalization,
ref.py:60-61) => backend 'gcs'. The os.path.exists 404 fast-fail is explicitly GATED behind
`input_ref.backend == 'local'` (main.py:283), so a bucket URI is NOT stat'd at submit.
create_job stores the RAW gs:// string (main.py:293, database.py:493).\\n\\n2. WORKER
(job_worker.py): claim_due_job -> get_job SELECT * round-trips the gs:// video_path;
_job_to_request rebuilds ProcessRequest(video_path=<gs://...>) verbatim (job_worker.py:64-69);
_run_claimed_job calls _execute_processing then set_job_video_id + mark_job_done
(job_worker.py:84-90).\\n\\n3. FETCH + PIPELINE (main.py:159-237): acquire_local_input downloads
the gs:// blob to scratch (storage/__init__.py:59-74 -> fetch -> gcs.py fetch_to_local:30-39),
computes video_id = MD5 of the fetched bytes, runs validators+segmenter on the LOCAL
path.\\n\\n4. DB WRITE (queryable): add_video UPSERTs the videos row unconditionally on
validation pass (main.py:190, database.py:287-304); the segmenter writes play_segments via
add_play_segment keyed by video_id (motion.py:201 etc., database.py:341-350). POST /api/query
reads them back (main.py:368). GET /api/jobs/{id} surfaces video_id (main.py:315), so the SPA
can deep-link ?video=<id>.\\n\\n5. The gs:// download itself needs google-cloud-storage + ADC at
run time ? these are ENVIRONMENTAL runtime prerequisites, not code gaps: the import is lazy and
a missing dep / auth failure raises inside the worker and is recorded as a loud JOB failure
(job_worker.py:95-97 mark_job_failed), never silently dropped. The submit still returns 202. So
the integration is fully wired in code; only the live GCS fetch is gated on the deploy
environment.\\n\\nNote: the videos.filepath column stores the ORIGINAL gs:// URI (main.py:190
passes req.video_path), while the local scratch copy is ephemeral (OS temp). This is intentional
per the M2 docstring (the source URI is kept for the DB record/report; only file-readers use the
resolved local path). A later /api/compile on that video_id would re-need the source
(video['filepath'] is the gs:// URI) ? but that is the compile path, outside this claim's scope
(process -> queryable video_id), which is fully satisfied."}]

```

TEST MAP: {"hasGcsProcessTest": true, "patternFile": "C:/Users/avidu/Projects/badminton-highlight-indexer/tests/test_api_endpoints.py", "patternSummary": "The best end-to-end pattern for a gs://-into-API-DB guard is the M2 block in tests/test_api_endpoints.py, specifically test_process_nonlocal_uri_skips_exists_and_fetches (lines 375-398). It is in-process (FastAPI TestClient, not subprocess) and drives a real job to completion deterministically via the shared

`env` fixture (tests/test_api_endpoints.py:27-35 and an identical copy in tests/test_async_jobs.py:34-42). The `env` fixture: builds a real Database(tmp_path/'t.db'); monkeypatch.setattr(m, 'db_instance', db); monkeypatch.setattr(m, 'global_config', AppConfig()); and crucially monkeypatch.setattr(m, '_get_executor', lambda: InlineExecutor()) where InlineExecutor.submit(fn, *a, **k) just calls fn() synchronously ? so POST /api/process returns 202+job_id and the job is ALREADY 'done' by the time GET /api/jobs/{id} is read (no threads, no polling, fully deterministic). The existing gs:// test monkeypatches the fetch seam (m.storage.fetch -> a fake that writes bytes to dst), a capturing segmenter (_CaptureSeg via m.segmenter_registry.get) to assert the segmenter received a LOCAL path (not the gs:// URI), and m.registry.run_all_with_context -> (_pass(), {}) to pass validation. It then asserts: r.status_code==202 (non-local skips the os.path.exists 404 gate), job['status']=='done', the segmenter saw a fetched local x.mp4 (not gs://), AND the API-DB record db.get_video(job['video_id'])['filepath'] keeps the SOURCE gs:// URI (provenance). The companion that drives a REAL ThreadPoolExecutor with a deadline/poll loop (closer to #281's liveness intent but still in-process) is test_real_executor_completes_job in tests/test_async_jobs.py:332-352. The actual #281 subprocess pattern lives in tests/test_server_entrypoint.py::test_python_m_backend_main_drains_submitted_jobs ? it Popen's `python -m backend.main --server --port ...`, waits on /api/health, POSTs /api/process, and polls /api/jobs/{id} asserting status != 'queued'; but it cannot inject fakes, so it only works with a real local file, not a hermetic gs:// fetch.", "mockStrategy": "Cleanest hermetic seam: monkeypatch.setattr(m.storage, 'fetch', fake_fetch) where fake_fetch(uri, dst=None, *, config=None, overwrite=False) writes fixture bytes to dst and returns dst. This is the EXACT seam the existing test_process_nonlocal_uri_skips_exists_and_fetches uses (tests/test_api_endpoints.py:380-385) and works because of the call chain: backend/main.py:159 calls storage.acquire_local_input(req.video_path, global_config.storage); acquire_local_input (backend/storage/_init_.py:59-74) resolves the ref, and for a non-local backend calls the BARE name `fetch(ref.uri, dst, config=sc)` at line 74 ? which binds to the `fetch` attribute of the backend.storage package module (defined at _init_.py:106). Because main.py imported `from backend import storage` and references m.storage.fetch, patching m.storage.fetch (== backend.storage.fetch) intercepts it before any GCS SDK or get_storage()/_backend_class('gcs') is touched ? so NO google-cloud-storage import, no network. Avoid patching at lower seams for this guard: GCSStorage(client=fake) (used by tests/test_storage.py:225 test_gcs_backend_roundtrip_with_fake_client) and a fake via get_storage/_backend_class are correct for unit-testing the GCS backend but would force importing backend.storage.gcs (and the real client construction path) and don't exercise the /api/process->DB flow. Patching m.storage.fetch is the single hermetic seam that keeps the whole pipeline real (resolve_input_ref, compute_video_id over the fetched bytes, validators, segmenter, db.add_video keeping the gs:// URI) while faking only the network download. Note: this in-process seam does NOT survive a subprocess, so a #281-style subprocess variant for gs:// is not hermetic without an injected/fake backend or a no-SDK scheme like gdrive:// over a tmp mount.", "testRunner": "python run_all_tests.py (repo root C:/Users/avidu/Projects/badminton-highlight-indexer/run_all_tests.py). It defaults pytest to -q and passes all extra args straight through (e.g. `python run\_all\_tests.py -k process`, `--cov=backend --cov-report=term-missing`); it shells out to `[sys.executable, '-m', 'pytest', \*args]` and returns pytest's own exit code. The suite is fully mocked/offline (no GPU/network/WSL). TestClient-based tests guard themselves with pytest.importorskip('httpx') at module top (tests/test_api_endpoints.py:10, tests/test_async_jobs.py:18), so they skip cleanly where httpx is absent ? install httpx so they actually run. obsreport is optional (reporting tests importorskip it). There is NO conftest.py anywhere in the repo (the shared `env` fixture is duplicated inline in each TestClient test file, not centralized).", "gaps": ["No conftest.py exists in the repo ? the `env` TestClient fixture (real Database + monkeypatched m.db_instance/m.global_config/m._get_executor=InlineExecutor) and helpers (_video, _pass, _CaptureSeg) are copy-pasted into both tests/test_api_endpoints.py and tests/test_async_jobs.py; a new B-P1 guard would either duplicate the fixture again or live in test_api_endpoints.py next to the existing M2 block to reuse it.", "An assertion already exists (test_process_nonlocal_uri_skips_exists_and_fetches): gs://-equivalent URI -> 202 (skips 404 gate) -> job done -> segmenter saw a LOCAL fetched path -> db.get_video(video_id)['filepath'] == the source gs:// URI. BUT it uses a gs://-style 'gs://bucket/videos/x.mp4' that is normalized via parse_uri's gs->gcs mapping; the test string is literally 'gs://...'. To 'lock B-P1' explicitly for BOTH gs:// AND gcs:// you'd want a parametrized assertion over ('gs://...', 'gcs://...') confirming both reach the API-DB with a queryable video_id (db.query_play_segments(video_id, {})) non-empty ? the current test asserts the source URI is kept and status==done but does NOT additionally assert query_play_segments for the gs:// case (that stronger DB-queryability assertion is only made for LOCAL inputs in test_job_done_with_segments_and_video_id).", "There is NO subprocess (#281-style) guard for the

gs:///gs:/// path. test_server_entrpoint.py only covers a real LOCAL dummy file. A true 'real regression guard like #281' for gs:/// cannot use the m.storage.fetch monkeypatch in a subprocess; the hermetic options for a subprocess variant are (a) a gdrive:/// URI over a tmp 'My Drive' mount (no SDK, see tests/test_storage.py:252) routed through --video-path, or (b) wiring an injectable fake backend via env/config ? neither exists yet.", "The existing gs:/// coverage asserts the segmenter received a local path and the DB kept the URI, but does NOT assert the returned job['video_id'] is the MD5 of the FETCHED bytes (compute_video_id(local_video)) ? worth pinning so a regression that hashes the URI string instead of the fetched file is caught."}]}

SPA MAP (B-P2): {"currentFlow": "The SPA has NO router. `App.tsx:48` reads the video id ONCE at mount: `const videoId = useMemo(() => new URLSearchParams(window.location.search).get(`\"video\"`), [])`. That single value drives all branching: it seeds `rallies/order` (DEMO_RALLIES if absent, [] if present) and `loadState` (App.tsx:50-52). The back-half: an effect keyed on `[videoId]` (App.tsx:61-85) calls `listRallies(videoId, cid)` = POST /api/query {video_id} ? returns play_segments ? setRallies/setOrder + sel.apply(all ids) + loadState `\"loaded\"`. A second effect (App.tsx:90-98) fetches GET /api/config for the min_shot_count floor. `onBuild` (App.tsx:109-142) does POST /api/compile {video_id, segment_ids, output_filename} via compileReel(), then builds the download/preview URL with highlightUrl(videoId, filename). The `\"back-half begins\"` the moment a non-null videoId exists ? everything from /api/query onward already works. The gap is purely the FRONT-half: nothing in the app ever SETS a video id (videoId is read-only from the URL and never written), so there is no path to get from `\"I have a bucket URI\"` to `\"?video=<id>\"`.\", \"apiClient\": \"All engine calls go through `web/src/api/client.ts`, a thin typed wrapper. The private `request<T>(path, opts)` (client.ts:31-59) does same-origin `fetch(`\"/api/\"+path)` with `content-type: application/json` + `x-correlation-id: cid` headers, logs `api.request`/`api.response`/`api.http_error`/`api.network_error`, and throws a typed `ApiError {status, body, cid}` on !res.ok or network failure. Public helpers wrap it: `listRallies` (POST /query), `compileReel` (POST /compile), `getConfig` (GET /config), plus pure `highlightUrl`/`minShotCount`. There is currently NO `processVideo` (POST /api/process) and NO `getJob` (GET /api/jobs/{id}) helper ? those two must be added. Types live in `web/src/api/types.ts` (RallySegment, QueryResponse, CompileResponse, EngineConfig); there is no ProcessResponse / JobStatus type yet. Dev wiring: `vite.config.ts:11-14` proxies `/api` ? http://localhost:8000; prod is one origin behind a reverse proxy.\"}, {\"filesToTouch\": [\"web/src/api/types.ts ? add ProcessRequest body shape + ProcessResponse {job_id, status, poll} + JobStatus {job_id, status: 'queued'|'running'|'done'|'failed', progress, video_id, error, result}\", \"web/src/api/client.ts ? add processVideo(videoPath, opts, cid) ? POST /api/process (202) and getJob(jobId, cid) ? GET /api/jobs/{job_id}\", \"web/src/hooks/useIngestJob.ts ? NEW hook: submit + poll loop with backoff, cancellable, returns {phase, progress, error, submit, reset} (mirrors useRallySelection.ts placement)\", \"web/src/components/IngestPanel.tsx ? NEW: bucket-URI input + Process button + live progress/status + error; on done calls onLoaded(video_id)\", \"web/src/App.tsx ? make videoId STATE not a one-shot useMemo (seed from URL, allow setVideoId); render IngestPanel when no videoId (above/instead of ModeBanner+QueryBar); onLoaded sets videoId + history.replaceState(`?video='+id` so reload is stable and the existing [videoId] effect fires\", \"web/src/i18n/locales/en/common.json ? add an `ingest.\*` block\", \"web/src/i18n/locales/hi/common.json ? same keys, Hindi\", \"web/src/i18n/locales/kn/common.json ? same keys, Kannada\", \"web/src/i18n/hygiene.test.ts ? already enforces key-parity across en/kn/hi, so all three must stay in lockstep\", \"web/src/components/IngestPanel.test.ts(x) + web/src/hooks/useIngestJob.test.ts ? NEW tests mirroring existing client.test.ts / *.test.ts conventions\"], \"i18nKeysNeeded\": [\"ingest.title\", \"ingest.intro\", \"ingest.uriLabel\", \"ingest.uriPlaceholder (e.g. gs://bucket/match.mp4)\", \"ingest.uriAriaLabel\", \"ingest.submit\", \"ingest.submitting\", \"ingest.queued\", \"ingest.running\", \"ingest.progress (interpolates {progress} free-text phase from the job, e.g. 'starting'|'finished')\", \"ingest.done\", \"ingest.failed (interpolates {error})\", \"ingest.emptyResult (job done but result.video_id absent / pipeline result.status='failed')\", \"ingest.invalidUri (client-side: empty/looks-not-a-uri)\", \"ingest.networkError (fetch threw ? engine unreachable)\", \"ingest.retry\", \"ingest.cancel\"], \"sketch\": \"Reuse the existing client+log+ApiError pattern exactly; do NOT hand-roll fetch.\\n\\n1) api/types.ts ? add: `interface ProcessResponse { job\_id: string; status: string; poll: string }` and `interface JobStatus { job\_id: string; status: 'queued'|'running'|'done'|'failed'; progress?: string|null; video\_id?: string|null; error?: string|null; result?: { status?: string; video\_id?: string } | null }`. (Confirmed against backend/main.py:297-322: POST returns 202 {job_id,status:'queued',poll}; GET returns {job_id,status,progress,video_id,error,result,reports,...}. Engine `status` ? queued|running|done|failed; on done the inner `result.status` ? success|failed and `result.video\_id` is the canonical id ? also surfaced top-level as job.video_id.)\\n\\n2)

api/client.ts ? add two helpers mirroring listRallies/compileReel:\n`export async function processVideo(videoPath: string, cid?: string): Promise<ProcessResponse> { return request<ProcessResponse>('/process', { method:'POST', body: JSON.stringify({ video\_path: videoPath }), cid }); }\n`export async function getJob(jobId: string, cid?: string): Promise<JobStatus> { return request<JobStatus>('/jobs/'+encodeURIComponent(jobId), { method:'GET', cid }); }\n` (ProcessRequest also supports optional player\_pool/max\_frames/segmenter per main.py:91-95 ? leave them out of the v1 minimal UI; send only video\_path.)\n\n3) hooks/useIngestJob.ts ? a small state machine, placed next to useRallySelection.ts:\n`phases: 'idle' | 'submitting' | 'polling' (carries job status+progress text) | 'done' (video_id) | 'error' (message). `submit(uri)` : newCorrelationId() (reuse lib/log) ? log 'ingest.submit_start' ? processVideo(uri, cid) ? on 202 start a poll loop on job_id with the SAME cid threaded through getJob so the whole journey is one correlatable id (matches the App.tsx onBuild/listRallies cid discipline). Poll loop: setTimeout-based (NOT setInterval) so each tick waits for the prior getJob to resolve; backoff e.g. 1s?1.5s??cap 5s; stop when status==='done' or 'failed'. On 'done': if result.status==='failed' or no video_id ? error('ingest.emptyResult'); else done(video_id). Errors: catch ApiError ? map via err.status (404 unknown job, 400 bad/unsupported URI scheme ? ingest.invalidUri using parseDetail from App.tsx, 5xx ? generic) and network throw ? 'ingest.networkError'. Return a cleanup that flips an `alive` flag (same idiom as App.tsx:80-82) so unmount/cancel kills the loop.\n\n4) components/IngestPanel.tsx ? structurally a sibling of QueryBar.tsx: useTranslation(); a <form onSubmit> with a text <input> (value/onChange, aria-label={t('ingest.uriAriaLabel')}, placeholder={t('ingest.uriPlaceholder')}) + a submit button disabled while phase==='submitting' || 'polling' (label flips t('ingest.submit')?t('ingest.submitting')); below it an aria-live='polite' region rendering the phase: queued/running show t('ingest.running') + the free-text job.progress via t('ingest.progress',{progress}); error shows the red banner style already used by ModeBanner/CompileResult (border-red-200 bg-red-50 text-red-800) with a Retry button. Props: `onLoaded(videoId: string)`. Client-side guard before submit: trim + reject empty (t('ingest.invalidUri')) ? keep it light; the engine does the authoritative scheme/path validation (main.py:271-290 ? 400, surfaced via ApiError body).\n\n5) App.tsx handoff ? the ONE structural change: turn videoId from a one-shot `useMemo` into state seeded from the URL: `const [videoId, setVideoId] = useState(() => new URLSearchParams(window.location.search).get('video'))`. The existing `useEffect([videoId])` (currently the load-rallies effect) then fires automatically when setVideoId is called ? no other change to the back-half. Render: when `!videoId` show <IngestPanel onLoaded={({id})=>{ setVideoId(id); window.history.replaceState(null, '', '?video='+encodeURIComponent(id)); }} /> in place of (or above) the demo ModeBanner; once videoId is set the existing query?grid?compile UI renders exactly as today and ?video= survives a reload. replaceState keeps the URL the single source of truth and reuses the proven [videoId] load path rather than inventing a parallel rally-loading code path.", "risks": ["Polling/backoff: use recursive setTimeout (await getJob each tick) not setInterval, else overlapping in-flight requests pile up if the engine is slow; cap backoff (~5s) and consider a max-attempts/elapsed ceiling so a wedged 'running' job doesn't poll forever. Thread one cid through submit + every poll (like onBuild) so the journey is traceable; log a loud 'ingest.poll_timeout' if the ceiling hits.", "Cancellation/unmount: the poll loop MUST honor an `alive` flag (App.tsx:80-82 idiom) and a user Cancel; a leaked loop keeps calling getJob after the user navigated to ?video=. StrictMode double-mounts effects in dev (main.tsx:14-17) ? guard against a double submit.", "Two status layers are easy to conflate: job.status (queued|running|done|failed = execution) vs result.status (success|failed = pipeline verdict). 'done' with result.status==='failed' or a null video_id is a SUCCESS-shaped HTTP 200 that is actually a failure ? must map to ingest.emptyResult, not silently hang or fake a load.", "Error mapping: reuse ApiError + parseDetail (currently a private fn in App.tsx ? extract to lib so IngestPanel can reuse it). 400 from /api/process is the engine's path/scheme validation (unsupported bucket scheme, out-of-allowed-root, missing local file ? 404) ? surface the engine's `detail`, don't show a generic message. Network throw (engine down) is the common non-tech-savvy case ? friendly t('ingest.networkError'), not a stack string.", "Offline-banner interplay: there is currently NO offline/navigator.onLine banner in the SPA (only self-contained assets + loud ApiError logging). The ingest screen is the FIRST screen a fresh user hits and it REQUIRES the engine ? so a friendly 'engine unreachable / is it running?' state matters more here than anywhere else. If an online/offline banner is added later, ensure it doesn't double-report with the IngestPanel's own networkError state.", "Long jobs vs UX expectation: real video processing can take minutes (per memory: cv2 extraction bottleneck, 4K stitch). The progress text from the engine is free-form TEXT ('starting'/'finished'), not a percentage ? the UI can only show phase words, so set expectations (indeterminate spinner + the phase word), don't imply a percent bar.", "i18n parity: hygiene.test.ts enforces identical key sets across en/kn/hi ? adding ingest.* to only en will fail the suite; all three catalogs must land together. kn/hi are AI-drafted/native-review-pending per i18n/index.ts, so flag the new strings for review.", "URL

handoff: replaceState (not a full reload) reuses the existing [videoId] effect, but the config-floor effect (App.tsx:90) runs once on mount ? confirm min_shot_count is still fetched for an ingest?load session (it is, since it's not keyed on videoId). Encode the id in the querystring (encodeURIComponent) to match how listRallies/highlightUrl already encode ids."}]}

ADVERSARIAL VERDICTS (0 of 3 skeptics refuted the claim): [{"lens": "SUBMIT-TIME GATE (backend/main.py process_video submit path: validate_input_path, resolve_input_ref ValueError mapping, input_ref.backend=="local" existence check, explicit-segmenter check)", "refuted": false, "reasoning": "I traced every branch of the submit path process_video() (backend/main.py:260-300) for req.video_path='gs://bucket/video.mp4', req.segmenter=None, under DEFAULT single-user config. Default config verified: config.json has NO `security` and NO `storage` sections, and no deployment.profile, so the loader's no-profile shallow merge (backend/config/loader.py:184) lets the Pydantic defaults supply SecurityConfig(allowed_video_root=None) and StorageConfig(input_backend='local', input_root=''); default_segmenter='gemini'. Gate-by-gate: (1) validate_input_path('gs://...', allowed_root=None) at main.py:271 -> only the null-byte check runs (backend/utils/paths.py:60-63); a gs:// URI has no null byte -> PASS, no 400. (2) resolve_input_ref at main.py:274 -> resolve_input_uri sees an explicit scheme (ref.py:87, rule 1) and returns it as-is; parse_uri normalizes scheme 'gs'-'>'gcs' (ref.py:60-61) and 'gcs' is in _KNOWN (ref.py:22,65-67), returning StorageRef(backend='gcs', bucket='bucket', key='video.mp4'). NO ValueError is raised, so the except ValueError -> HTTPException(400) at main.py:275-276 never fires. (3) main.py:283 `if input\_ref.backend == 'local' and not os.path.exists(...): backend is 'gcs', so the AND short-circuits and the 404 (main.py:284) is SKIPPED ? exactly as the code comment at main.py:277-282 intends (a bucket URI is not stat'd at submit). (4) main.py:285 `if req.segmenter and not segmenter_registry.get(...): req.segmenter is None (falsy), so the unknown-segmenter 400 is SKIPPED. After all four gates the endpoint calls create_job (which inserts unconditionally, inspecting nothing about the URI ? backend/infrastructure/database.py:486-499) and returns 202 + a queryable job_id. The downstream worker then resolves the same gs:// (acquire_local_input, main.py:159) and computes/persists video_id. I searched for any submit-path branch that 400s/404s a valid gs:// URI under default config and found NONE. The only failure modes are environmental (missing google-cloud-storage SDK or ADC at the worker fetch), explicitly excluded by the lens, and they occur on the worker, not the submit path. No concrete code-gate counterexample exists; the claim's submit-time portion holds.", "counterexample": ""}], {"lens": "WORKER/EXECUTION + DB WRITE (post-202: job_worker drain, _job_to_request, _execute_processing/acquire_local_input, add_video/get_video persistence)", "refuted": false, "reasoning": "I traced the entire post-202 worker/DB-write path and found NO code gate that makes a valid gs://|gcs:// URI fail to yield a queryable API-DB video_id under default single-user config (allowed_video_root=None, input_backend='local', input_root='').\n\nSubmit acceptance: backend/main.py:269-296 ? validate_input_path (utils/paths.py:53-63) only checks null bytes when allowed_root is None (default), so gs:// passes; storage.resolve_input_ref returns a gcs-backend ref (gs:// is normalized to gcs:// in storage/ref.py:60-61); the local-existence os.path.exists fast-fail at main.py:283 is SKIPPED because input_ref.backend != 'local'; create_job persists the row 'queued'; main.py:296 `\_get\_executor().submit(\_drain\_due\_jobs)` actually fires the worker by default.\n\nWorker drain: job_worker.py:112 claim_due_job -> :116 _run_claimed_job -> :84 _job_to_request rebuilds ProcessRequest from the stored video_path (the gs:// URI verbatim, identical handling to a local path).\n\n_execute_processing (main.py:140-258): line 159 acquire_local_input(req.video_path, global_config.storage) -> storage/__init__.py:59-74 resolves the gcs ref, mkdtemp's a scratch dir, and fetch()->GCSStorage.fetch_to_local (gcs.py:30-39) downloads to scratch and returns a LOCAL path. dst is a valid path: ref.name yields 'x.mp4' (ref.py:44-45). Line 160 compute_video_id(local_video) hashes the FETCHED bytes (hashing.py opens the local file), so the id is well-formed exactly as in the local case. add_video(video_id, req.video_path, ...) is invoked on every terminal branch (validation-fail main.py:179, processed main.py:190) ? it UPSERTs a row keyed on id (database.py:287-304). Storing the gs:// URI in the text `filepath` column is harmless to keying.\n\nget_video (database.py:418-425) selects WHERE id=? and returns the row irrespective of the filepath column value, so db.get_video(video_id) returns a queryable row. The worker also set_job_video_id + mark_job_done, surfaced via /api/jobs/{job_id}.video_id (main.py:303-322).\n\nThe ONLY gs:// vs local divergence in this path is the fetch-to-scratch network download, whose failure modes are a missing google-cloud-storage dep or missing ADC ? explicitly excluded as environmental, not a code gate. The DB write is keyed on the content hash and is byte-for-byte identical between the two cases. No concrete counterexample exists for my lens.\n\nCaveat (out of my lens, does not refute): the shipped SPA in backend/api/static/app.js has no ?video= query-param loader (no URLSearchParams/location.search), so the 'SPA can load via ?video=' sub-claim is unsupported by the actual frontend ? but that is the frontend lens and does not affect whether get_video(video_id) returns a queryable API-DB row, which it

does.", "counterexample": ""}, {"lens": "CONFIG & EDGE CASES (input_backend default, gs://?gcs:// normalization, allowed_video_root default, gdrive://, and SPA ?video= back-half)", "refuted": false, "reasoning": "I traced the full path under DEFAULT single-user config (storage.input_backend=\"local\", input_root=\"\", security.allowed_video_root=None ? confirmed in backend/config/models.py:283,315-316) and found NO code gate that fails a valid gs://|gcs:// URI.\n\nROUTING / ACCEPTANCE (POST /api/process, backend/main.py:260-300):\n- backend/utls/paths.py:53-63 validate_input_path: with allowed_root falsy (default), ONLY the null-byte check runs, so gs:// passes. (A *configured* allowed_video_root would reject a non-local URI via resolve_under_root ? but that is non-default; the lens asks about default config, where it is None.)\n- backend/storage/ref.py:87-88 resolve_input_uri precedence rule 1: an explicit scheme:// is returned AS-IS *before* input_backend/input_root are consulted ? so the default input_backend=\"local\" is irrelevant for an explicit gs://. The URI routes to gcs regardless.\n- backend/storage/ref.py:60-61 parse_uri: scheme is lowercased and gs?gcs normalized; both gs:// and gcs:// produce an identical gcs StorageRef with non-empty bucket+key. No edge here.\n- backend/main.py:283: the local-existence os.path.exists fast-fail is guarded by `input\_ref.backend == \"local\"`, so it is correctly SKIPPED for the gcs backend (no false 404). Job row is created (create_job) and 202 returned.\n\nBACK-HALF (worker, backend/main.py:140-257 + backend/storage/__init__.py:59-74):\n- acquire_local_input resolves the gcs ref, makes a scratch dir, and calls fetch ? GCSStorage.fetch_to_local (backend/storage/gcs.py:30-39), which downloads to dst. dst is always supplied (os.path.join(scratch, ref.name or \"in.mp4\")); ref.name for gs://bucket/videos/x.mp4 = \"x.mp4\". No code gate ? the only failure modes are the lazy `from google.cloud import storage` import (gcs.py:24) or missing ADC, both ENVIRONMENTAL and explicitly excluded.\n- compute_video_id hashes the fetched local file; add_video (main.py:190) writes the row with the ORIGINAL gs:// string as the path column and the MD5 as id; get_video (backend/infrastructure/database.py:418-425) returns that row by id.\n\nSPA ?video= back-half: the engine's bundled SPA (backend/api/static/app.js) has no ?video= bootstrap, but the actual ?video=-consuming SPA is the khelsutra web app (C:/Users/avidu/Projects/khelsutra/web/src/App.tsx:48 reads ?video=<id>; :66 listRallies(videoId) ? POST /api/query keyed purely by video_id). It loads strictly by the MD5 video_id and never touches the source-path scheme, so a gs://-sourced video is loadable byte-identically to a local one. gdrive:// follows the same non-local path (resolve?fetch?hash?row) with no extra gate.\n\nNo realistic DEFAULT config silently fails gs://?queryable video_id on a code path; all failures are environmental (SDK/auth), which the task excludes.", "counterexample": ""}]

Decide B-P1's final verdict. If the claim survives (skeptics found no real code gate), the B-P1 PR is: (a) a #281-style regression test locking gs://?/api/process?API-DB video_id (hermetic GCS fetch fake), and (b) an HONEST correction of khelsutra/docs/DRY_RUN_CUJ.md §1/§2 (which currently calls this 'net-new'). If a skeptic found a real gap, specify the minimal code fix. Give the concrete test file+cases, the exact doc edits, the consolidated B-P2 sketch for the follow-up PR, and droplet (option 3) deploy notes. Return structured JSON.

2. assistant (2026-06-21T09:22:07.826Z)

I'll synthesize the final plan for B-P1. Let me first verify the key claims against the actual code and docs before committing to a verdict.

3. user (2026-06-21T09:22:10.921Z)

Your configuration specifies to merge with the ref 'refs/heads/feat/serving-m4-device-context' from the remote, but no such ref was fetched.

4. user (2026-06-21T09:22:12.572Z)

Already up to date.

5. assistant (2026-06-21T09:22:18.295Z)

The branch tracking ref isn't on remote, that's fine. Let me verify the critical code claims directly. I'll check the submit path, the existing gs:// test, and the DRY_RUN_CUJ doc.

6. user (2026-06-21T09:22:19.775Z)

```

259
260 @app.post("/api/process", status_code=202)
261 def process_video(req: ProcessRequest):
262     """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.
263
264     Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
265     client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
266     runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
267     worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
268     """
269     allowed_root = getattr(getattr(global_config, "security", None),
"allowed_video_root", None)
270     try:
271         validate_input_path(req.video_path, allowed_root=allowed_root)
272         # M2: resolve the input once (path or storage URI) ? raises ValueError for an
unsupported
273         # scheme, which we map to a clean 400 (an unknown scheme must never 500).
274         input_ref = storage.resolve_input_ref(req.video_path, getattr(global_config,
"storage", None))
275     except ValueError as e:
276         raise HTTPException(status_code=400, detail=str(e)) from e
277     # The local-existence fast-fail applies only to a LOCAL input, and stats the
RESOLVED local path
278     # (input_ref.key ? e.g. local://? stripped to the bare path), not the raw URI
string. A
279     # bucket/Drive URI can't be cheaply os.path.exists()'d ? its existence is verified
when the worker
280     # fetches it (a fetch miss fails the job loudly). validate_input_path above still
runs for every
281     # input: its null-byte guard always applies, and a configured allowed_video_root
(hosted mode)
282     # rejects a non-local URI as out-of-root.
283     if input_ref.backend == "local" and not os.path.exists(input_ref.key):
284         raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")
285     if req.segmenter and not segmenter_registry.get(req.segmenter):
286         available = list(segmenter_registry.list_available().keys())
287         raise HTTPException(
288             status_code=400,
289             detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}")
290     )
291
292     job_id = uuid.uuid4().hex
293     if not db_instance.create_job(job_id, req.video_path, req.segmenter,
294                                req.player_pool, req.max_frames):
295         raise HTTPException(status_code=500, detail="Failed to persist job record.")
296     _get_executor().submit(_drain_due_jobs)
297     return JSONResponse(
298         status_code=202,
299         content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
300     )
301
302
303 @app.get("/api/jobs/{job_id}")
304 def get_job(job_id: str):
305     """Job state: {status, progress, result, reports}. `status` = execution state
306     (queued|running|done|failed); the pipeline verdict is result["status"]."""
307     job = db_instance.get_job(job_id)
308     if not job:
309         raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
310     result = job.get("result")
311     return {
312         "job_id": job["id"],
313         "status": job["status"],

```



```

314         "progress": job.get("progress"),
315         "video_id": job.get("video_id"),
316         "error": job.get("error"),
317         "result": result,
318         "reports": (result or {}).get("reports"),
319         "created_at": job.get("created_at"),
320         "started_at": job.get("started_at"),
321         "finished_at": job.get("finished_at"),
322     }
323
324     # --- Chunked upload (B2, PR-2) -----
325     # The chunked-upload island (in-process registry + helpers + the 4 endpoints) now lives
326     # in
327     # backend.api.uploads as a self-contained APIRouter. It resolves the live startup config
328     # via
329     # backend.main.global_config, so behavior ? route paths, status codes, body-size limits,

```

7. user (2026-06-21T09:22:20.207Z)

```

1     """TestClient coverage for the FastAPI seam (previously zero ? the compiler SyntaxError
2     that broke `import backend.main` shipped green because nothing imported it)."""
3     import os
4     import subprocess
5     import sys
6     from unittest.mock import patch, MagicMock
7
8     import pytest
9
10    pytest.importorskip("httpx") # TestClient needs httpx; skip cleanly where absent
11    from fastapi.testclient import TestClient
12
13    import backend.main as m
14    from backend.infrastructure.database import Database
15    from backend.config.models import AppConfig
16    from backend.pipeline.validators.base import ValidationResult
17
18
19    class _InlineExecutor:
20        """submit() runs the fn synchronously ? async jobs complete before the response
21        is inspected (full lifecycle coverage lives in test_async_jobs.py)."""
22
23        def submit(self, fn, *args, **kwargs):
24            fn(*args, **kwargs)
25
26
27    @pytest.fixture
28    def env(tmp_path, monkeypatch):
29        db = Database(str(tmp_path / "t.db"))
30        cfg = AppConfig()
31        monkeypatch.setattr(m, "db_instance", db)
32        monkeypatch.setattr(m, "global_config", cfg)
33        monkeypatch.setattr(m, "_get_executor", lambda: _InlineExecutor())
34        client = TestClient(m.app)
35        return client, db, tmp_path, monkeypatch
36
37
38    def _video(tmp_path, name="m.mp4"):
39        p = tmp_path / name
40        p.write_bytes(b"not-a-real-video-but-hashable")
41        return p
42
43
44    def _pass():
45        return [ValidationResult(validator_name="x", passed=True, message="ok", details={})]
46

```

```

47
48     def _fail():
49         return [ValidationResult(validator_name="blur", passed=False, message="blurry",
details={})]
50
51
52     class _FakeSeg:
53         def __init__(self, db, cfg):
54             self.db = db
55
56         def process_video(self, video_id, video_path, *a, **k):
57             sid = self.db.add_play_segment(video_id, 0.0, 5.0)
58             return [{"id": sid, "start_time": 0.0, "end_time": 5.0, "duration": 5.0,
"metadata": {}}]
59
60
61     def test_api_config_ok(env):
62         client = env[0]
63         r = client.get("/api/config")
64         assert r.status_code == 200
65         assert r.json()["sport"] == "badminton"
66
67
68     def test_process_success_with_mocked_segmenter(env):
69         """#16: submit returns 202 + job_id; the (inline) job carries the old response."""
70         client, db, tmp_path, mp = env
71         vid_file = _video(tmp_path)
72         mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
73         mp.setattr(m.segmenter_registry, "get", lambda name: _FakeSeg)
74         r = client.post("/api/process", json={"video_path": str(vid_file), "segmenter":
"x"})
75         assert r.status_code == 202
76         job = client.get(f"/api/jobs/{r.json()['job_id']}").json()
77         assert job["status"] == "done"
78         assert job["result"]["status"] == "success"
79         assert len(job["result"]["play_segments"]) == 1
80
81
82     def test_process_validation_failure_preserves_prior_segments(env):
83         client, db, tmp_path, mp = env
84         vid_file = _video(tmp_path)
85         vid = m.compute_video_id(str(vid_file))
86         db.add_video(vid, str(vid_file), "processed", [])
87         db.add_play_segment(vid, 1.0, 2.0) # prior good result
88         mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_fail(), {}))
89         r = client.post("/api/process", json={"video_path": str(vid_file)})
90         assert r.status_code == 202
91         job = client.get(f"/api/jobs/{r.json()['job_id']}").json()
92         assert job["status"] == "done" # worker finished; the PIPELINE verdict failed
93         assert job["result"]["status"] == "failed"
94         # PR-C: a failed re-validation must NOT cascade-delete prior segments.
95         assert len(db.query_play_segments(vid, {})) == 1
96
97
98     def test_process_rejects_null_byte_path(env):
99         client = env[0]
100         r = client.post("/api/process", json={"video_path": "a\x00b.mp4"})

```

8. assistant (2026-06-21T09:22:23.112Z)

The submit path and test fixture confirm the trace. Now let me read the existing gs:// test in detail and the DRY_RUN_CUJ doc.

9. user (2026-06-21T09:22:24.059Z)

```

360     mp.setattr(m.segmenter_registry, "get", lambda name: _CaptureSeg)
361     _CaptureSeg.last_path = None
362     r = client.post("/api/process", json={"video_path": str(vid), "segmenter": "x"})
363     assert r.status_code == 202
364     assert _CaptureSeg.last_path == str(vid)
365
366
367     def test_process_local_missing_file_still_404(env):
368         """A LOCAL input that doesn't exist still fast-fails 404 ? the existence gate is
preserved
369         for local paths (only non-local URIs skip it)."""
370         client = env[0]
371         r = client.post("/api/process", json={"video_path": "/no/such/local/file.mp4"})
372         assert r.status_code == 404
373
374
375     def test_process_nonlocal_uri_skips_exists_and_fetches(env):
376         """A gs:// input is NOT 404'd at submit (can't os.path.exists a bucket object); the
worker
377         fetches it to a local scratch file and the segmenter sees that LOCAL path, not the
URI."""
378         client, db, tmp_path, mp = env
379
380         def fake_fetch(uri, dst=None, *, config=None, overwrite=False):
381             with open(dst, "wb") as fh:
382                 fh.write(b"fetches-video-bytes")
383             return dst
384
385         mp.setattr(m.storage, "fetch", fake_fetch)
386         mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
387         mp.setattr(m.segmenter_registry, "get", lambda name: _CaptureSeg)
388         _CaptureSeg.last_path = None
389         r = client.post("/api/process", json={"video_path": "gs://bucket/videos/x.mp4",
"segmenter": "x"})
390         assert r.status_code == 202                                # NOT 404 ? non-local skips
the exists gate
391         job = client.get(f"/api/jobs/{r.json()['job_id']}").json()
392         assert job["status"] == "done"
393         assert _CaptureSeg.last_path is not None
394         assert not _CaptureSeg.last_path.startswith("gs://")      # a fetched LOCAL path, not
the URI
395         assert os.path.basename(_CaptureSeg.last_path) == "x.mp4"
396         # Provenance: the DB record keeps the SOURCE URI (not the ephemeral scratch path).
397         rec = db.get_video(job["video_id"])
398         assert rec is not None and rec["filepath"] == "gs://bucket/videos/x.mp4"
399
400
401     def test_process_unknown_scheme_returns_400(env):
402         """An unsupported scheme (https/ftp/file) is a clean 400, never a 500 ? parse_uri's
ValueError
403         must not escape the request handler."""
404         client = env[0]
405         for bad in ("https://h/x.mp4", "ftp://h/x.mp4", "file:///etc/passwd"):
406             r = client.post("/api/process", json={"video_path": bad})
407             assert r.status_code == 400, f"{bad} -> {r.status_code}"
408
409
410     def test_process_local_scheme_uri_accepted(env):
411         """An explicit local://<existing file> resolves to the bare path: the existence gate
stats the
412         RESOLVED key (not the raw URI) so it is accepted, and the segmenter sees the
stripped path."""
413         client, db, tmp_path, mp = env
414         vid = _video(tmp_path, "ls.mp4")
415         mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))

```

```
416         mp.setattr(m.segmenter_registry, "get", lambda name: _CaptureSeg)
417         _CaptureSeg.last_path = None
418         # Forward-slash local:// URI (parse strips the scheme to the bare path).
419         uri = "local://" + str(vid).replace("\\", "/")
```

10. user (2026-06-21T09:22:24.690Z)

No files found

11. assistant (2026-06-21T09:22:26.998Z)

The DRY_RUN_CUJ.md doesn't exist at that path. Let me find it and check the _CaptureSeg helper.

12. user (2026-06-21T09:22:27.647Z)

No files found

13. user (2026-06-21T09:22:28.601Z)

No files found

14. user (2026-06-21T09:22:29.626Z)

```
341: class _CaptureSeg:
349:     _CaptureSeg.last_path = video_path
360:     mp.setattr(m.segmenter_registry, "get", lambda name: _CaptureSeg)
361:     _CaptureSeg.last_path = None
364:     assert _CaptureSeg.last_path == str(vid)
387:     mp.setattr(m.segmenter_registry, "get", lambda name: _CaptureSeg)
388:     _CaptureSeg.last_path = None
393:     assert _CaptureSeg.last_path is not None
394:     assert not _CaptureSeg.last_path.startswith("gs://")    # a fetched LOCAL path, not the
URI
395:     assert os.path.basename(_CaptureSeg.last_path) == "x.mp4"
416:     mp.setattr(m.segmenter_registry, "get", lambda name: _CaptureSeg)
417:     _CaptureSeg.last_path = None
422:     assert _CaptureSeg.last_path == str(vid).replace("\\", "/")
462:     monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _CaptureSeg)
470:     _CaptureSeg.last_path = None
474:     assert _CaptureSeg.last_path is not None
475:     assert not _CaptureSeg.last_path.startswith("gs://")    # fetched LOCAL path
476:     assert os.path.basename(_CaptureSeg.last_path) == "cli.mp4"
477:     vid = m.compute_video_id(_CaptureSeg.last_path)
```

15. assistant (2026-06-21T09:22:32.494Z)

The DRY_RUN_CUJ.md doesn't exist yet in khelsutra. Let me look at the khelsutra docs directory structure and the _CaptureSeg helper, plus check whether the doc lives elsewhere or is net-new.

16. user (2026-06-21T09:22:35.195Z)

```
341     class _CaptureSeg:
342         """Records the video_path the segmenter actually received (M2: must be a LOCAL
path)."""
343         last_path = None
344
345         def __init__(self, db, cfg):
346             self.db = db
347
348         def process_video(self, video_id, video_path, *a, **k):
349             _CaptureSeg.last_path = video_path
350             sid = self.db.add_play_segment(video_id, 0.0, 5.0)
351             return [{"id": sid, "start_time": 0.0, "end_time": 5.0, "duration": 5.0,
"metadata": {}}]
```

```

352
353
354     def test_process_local_input_passes_identity_path(env):
355         """Default (local) input must reach the segmenter as the EXACT same path ? identity,
no copy
356         (the M2 zero-behaviour-change guarantee for today's local workflow)."""
357         client, db, tmp_path, mp = env
358         vid = _video(tmp_path)
359         mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
360         mp.setattr(m.segmenter_registry, "get", lambda name: _CaptureSeg)

```

17. user (2026-06-21T09:22:38.431Z)

```

total 140
drwxr-xr-x 1 avidu 197609    0 Jun 21 14:36 .
drwxr-xr-x 1 avidu 197609    0 Jun 21 12:12 ..
-rw-r--r-- 1 avidu 197609 16152 Jun 20 20:24 ANNOTATE_AND_TRAIN.md
-rw-r--r-- 1 avidu 197609  8491 Jun 21 14:36 DRY_RUN_CUJ.md
-rw-r--r-- 1 avidu 197609 26770 Jun 21 13:00 FRIENDLY_AND_MULTILINGUAL.md
-rw-r--r-- 1 avidu 197609 28650 Jun 20 18:28 LOCAL_APPLIANCE_ARCHITECTURE.md
-rw-r--r-- 1 avidu 197609 27874 Jun 20 23:14 PER_RALLY_SELECTION.md
-rw-r--r-- 1 avidu 197609  4956 Jun 20 18:40 PORTABILITY.md
-rw-r--r-- 1 avidu 197609  2051 Jun 21 13:55 README.md
-rw-r--r-- 1 avidu 197609  6948 Jun 20 11:30 REQUEST_ACCESS_FORM.md
---ROOT---
total 38
drwxr-xr-x 1 avidu 197609    0 Jun 21 12:12 .
drwxr-xr-x 1 avidu 197609    0 Jun 21 14:35 ..
drwxr-xr-x 1 avidu 197609    0 Jun 21 10:30 .claude
drwxr-xr-x 1 avidu 197609    0 Jun 21 14:52 .git
-rw-r--r-- 1 avidu 197609  297 Jun 20 23:27 .gitattributes
drwxr-xr-x 1 avidu 197609    0 Jun 20 12:43 .github
-rw-r--r-- 1 avidu 197609   78 Jun 20 18:40 .gitignore
-rw-r--r-- 1 avidu 197609 6617 Jun 21 12:12 NEXT_STEPS.md
-rw-r--r-- 1 avidu 197609  925 Jun 20 08:37 README.md
drwxr-xr-x 1 avidu 197609    0 Jun 21 14:36 docs
drwxr-xr-x 1 avidu 197609    0 Jun 21 12:12 site
drwxr-xr-x 1 avidu 197609    0 Jun 21 12:31 web
---GREP CUJ---
./docs/DRY_RUN_CUJ.md
./docs/LOCAL_APPLIANCE_ARCHITECTURE.md
./docs/PER_RALLY_SELECTION.md
./docs/README.md
./NEXT_STEPS.md

```

18. assistant (2026-06-21T09:22:41.582Z)

The DRY_RUN_CUJ.md does exist (16152 bytes wait, no ? it says 8491). Let me read it fully, focusing on \$1/\$2/\$6 which the task says call B-P1 "net-new".

19. user (2026-06-21T09:22:42.171Z)

```

1     # Dry-run CUJ ? UI drives the tool on a bucket video ? reel downloaded local
(multilingual)
2
3     > **Status:** `DRAFT` ? owner-gated . **Owner:** Avi Dullu . **Created:** 2026-06-21 .
**Goal for this session.**
4     > **Umbrella:** a milestone of `docs/LOCAL_APPLIANCE_ARCHITECTURE.md` (the appliance) +
`docs/PER_RALLY_SELECTION.md` (the shipped SPA back-half).
5     > **Honesty note:** grounded in a file:line integration-surface map (this session);
claims tagged `[verified]` / `[gap]`.
6
7     ## 0. The goal (owner's words)
8     Operate the **web UI** ? point at a **video on the GCS bucket** ? the engine (tool)

```

processes it ? **review/select rallies in the UI** ? **compile** ? **download the reel to the local machine**. Then **repeat with the UI in Hindi** to prove the multilingual workflow.

9

10 **## 1. What already works vs the gap** `[verified]`

11 **?? Back half (shipped ? PER_RALLY_SELECTION M0):** the SPA loads `?video=<id>` ? `POST /api/query` (rallies, default all-selected) ? deterministic query/select ? `POST /api/compile` ? `` preview + **??<a download>** saves the MP4 locally. Reel download-to-local **works today** (`GET /api/highlights/{video\_id}/{filename}` streams it; `/api/compile` also returns `download\_url`). Fully **en/kn/hi** (the switcher / `?lang=`).

12

13 **?? Front half (the gap):**

14 - The SPA has **no ingest/process UI** ? it can't point at a video (bucket or local), can't call `/api/process`, can't poll `/api/jobs`. It only gets a `video\_id` from `?video=` in the URL.

15 - **No HTTP bucket-ingest:** `POST /api/process` accepts only a **local** `video\_path`. The worker CLI (`python -m backend.worker infer --video-uri gcs://`) **can** fetch+process a bucket video (GCS is a real `StorageBackend`), **but** it writes `intervals.csv`/`report.json` to the bucket ? **NOT** to the API's SQLite `videos`/`play\_segments`. So a worker run alone does not give the SPA a queryable `video\_id`. **Bridging** "gcs://" ? a queryable `video\_id` in the API DB" is **net-new** (appliance T12/T14).

16

17 **## 2. Two paths to the goal**

18 **Path A ? Cheapest dry run NOW (~zero new code).** Pre-process the bucket video out-of-band to mint a `video\_id` in the engine DB, then drive the existing SPA back-half. Proves the **multilingual UI-driven workflow** end-to-end. The only non-UI step is "point at the bucket" (a one-line CLI pre-process).

19

20 **Path B ? Full "point at a bucket video IN the UI" (bigger, multi-session).** Build the in-UI ingest: a **net-new HTTP bucket-ingest endpoint** (`gcs://` ? process ? DB `video\_id`) + the SPA **ingest + progress** screens (appliance S1/S2). The proper appliance front-half.

21

22 **## 3. Path A runbook (the cheapest dry run) [verified-steps]**

23 1. **Engine host + deps:** run the engine FastAPI (`python -m backend.main --server`, binds `127.0.0.1:8000`) on a box with `ffmpeg` + OpenCV. Gemini/motion paths are **CPU-only**, no WSL ? local Windows works; the CPU droplet works too. (GPU/WSL only needed for native WASB.)

24 2. **Get the bucket video local + into the API DB:**

25 `gsutil cp gs://<bucket>/<match>.mp4 ./incoming/` (or the worker fetch), then `POST /api/process {"video\_path": "<abs local path>", "segmenter": "gemini"}` ? poll `GET /api/jobs/{job\_id}` until `result.video\_id`. *(`gemini` needs `GEMINI\_API\_KEY`; `motion` is key-free but hides fabricated metadata.)*

26 3. **Open the SPA against the engine:** `npm run dev` (Vite proxies `/api` ? `:8000`, same-origin) ? `http://localhost:5173/?video=<video\_id>` ? rallies auto-load.

27 4. **Select ? Build ? download:** pick rallies ? Build ? `` preview ? click the download link ? MP4 saved local.

28 5. **Repeat in Hindi:** click **??????** (or `?lang=hi`) ? redo select?compile?download ? confirm the whole flow works with the UI in Hindi.

29

30 **Small unblocked SPA polish** that makes Path A more "via the UI" (independent of A/B, useful in both): a **"Load a video by ID"** input (paste the `video\_id` ? load, instead of editing the URL) + consume the engine's `download\_url` (instead of hand-building it). One tight PR.

31

32 **## 4. Open decisions ? OWNER (these gate the dry run)**

33 1. **Scope:** Path A (cheapest dry run now) or Path B (build the full in-UI bucket ingest)? **Recommend A first** (proves the multilingual workflow fast), then B as the appliance build.

34 2. **Compute / detector:** **Gemini** (real rallies; needs your `GEMINI\_API\_KEY`; uploads downsized chunks to Google = paid + cloud) . **motion** (key-free, runs anywhere, but lower-quality boundaries + fabricated server/shot metadata stays hidden) . **GPU WASB** (best quality, but needs a GPU host + is owner-parity-gated, not production-verified). **Recommend Gemini** for "real-ish, cheapest" ? pending your key + OK (paid + uploads footage).*

35 3. **Engine host:** local Windows (simplest, Gemini/motion) . the CPU droplet `168.144.126.176` (shared testbed ? careful) . a GPU VM (only for WASB).

36 4. **Which bucket video:** e.g. `gs://khealsutra-rally-corporus/videos/mahadevpura\_smoke\_180s.mp4` (already staged, known-good) or another clean ?720p

```

clip (must pass the engine validators).
37
38     ## 5. Honest caveats
39     - **Gemini = paid + uploads footage to Google** (an external/paid action ? needs your
explicit OK + a clean clip). **Rotate the chat-shared `GEMINI_API_KEY`.**
40     - **motion** fabricates `server`/`receiver`/`shot_count` (the SPA correctly hides them)
? boundaries are real but metadata isn't.
41     - **GPU/native WASB** is partially wired + owner-byte-parity-gated ? **not production-
verified**.
42     - Path A's "point at the bucket" is a **CLI pre-step**, not in-UI ? full in-UI ingest is
Path B.
43     - Edge auth: none in the engine (localhost dry-run is fine; required before any non-
localhost exposure).
44
45     ## 6. Definition of done (the dry run)
46     - [x] A bucket video ? (engine processes) ? the **SPA loads its rallies** ? select ?
compile ? **reel downloaded to the local machine**, all driven from the UI (Path A allows a one-
line CLI pre-process). ? ? **Path A done (local, 2026-06-21)**, see §7.
47     - [x] The **same flow works with the UI in Hindi** ? multilingual workflow confirmed. ?
? (Kannada also works via the same switcher.)
48     - [ ] *(Path B, if chosen)* the bucket video is pointed-at **from the UI** (no CLI pre-
step).
49     - [ ] *(option 3)* the same CUJ confirmed on the **CPU droplet** (Linux deploy target) ?
droplet reachable but currently **un-provisioned** (no engine checkout; ~3.8GB RAM) ? needs a
from-scratch engine install or a proper deploy of the fixed engine.
50
51     ## 7. ? Run result ? Path A, local (2026-06-21)
52     Ran the real CUJ end to end on the local Windows box, isolated engine off
`origin/master` (fresh DB/output worktree), **Gemini** detector:
53     1. `gs://khelsutra-rally-corpus/videos/mahadevpura_smoke_180s.mp4` (1080p/30fps) ?
`gsutil cp` local ? `POST /api/process {segmenter: gemini}`.
54     2. Engine processed via Gemini (3 chunks) ? **12 rallies** (16 found, 4 too-short
rejected), `video_id 05e0e577?`.
55     3. SPA (`npm run dev`, Vite proxies `/api`??:8000) opened at `?video=05e0e577?` ?
**loaded the 12 real rallies** ("Loaded 12 rallies from this session").
56     4. **Build ? `/api/compile` ? reel** (1920x1080, 76.5s) ? **downloaded to
`~/Downloads/khelsutra-dryrun/highlights_local_en.mp4`** (50.9 MB).
57     5. **Repeated with the UI in Hindi** (`?lang=hi`): the page + rally cards + "?? ?????"
+ "???? ???? ??" all in Hindi; reel compiled + **downloaded** (`highlights_local_hi.mp4`).
? multilingual workflow confirmed.
58
59     Screencast/proof: per-locale screenshots; both reels on the local machine. Screencast of
the i18n SPA is at `gs://khelsutra-rally-corpus/promo/khelsutra-i18n-screencast.mp4`.
60
61     ### ? Fix the dry run surfaced (engine) ? MERGED-PENDING
62     The async-job server **hung every job in `queued`** when run the documented way (`python
-m backend.main`): main.py runs as `__main__` (sets `__main__.db_instance`), but `job_worker`
reads `import backend.main` ? a **separate** module whose `db_instance` is `None` ? the drain
crashes silently. Latent because the unit tests monkeypatch the executor inline and the UI was
verified vs a stub engine. **Fix + a real-subprocess regression test ? engine
[#281](https://github.com/Khelsutra/badminton-highlight-indexer/pull/281)** (dispatch through
`backend.main`; FAILS-without / PASSES-with). Unblocked the run via the single-module
invocation; the fix applies to the droplet too.
63

```

20. assistant (2026-06-21T09:22:48.462Z)

Now I have the full picture. The doc's §1 and §2 are the key claims. The trace plus all 3 skeptics confirm: B-P1 (gcs:// ? /api/process ? queryable API-DB video_id) **already works in code** under default config. The doc §1 (line 15) and §2 (line 18/20) currently claim this bridging is "net-new" ? which is FALSE per the verified trace.

Let me verify the remaining critical claims: that `add\_video` is unconditional and `query\_play\_segments`/`compute\_video\_id` work as the trace says, and confirm the CLI test that drives a real subprocess for the local path. Let me also check the worker endpoint test and

the `set\_job\_video\_id` flow.

21. user (2026-06-21T09:22:49.025Z)

```
140     def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
141         """The full hash ? validate ? segment ? report pipeline for one video.
142
143         This is the former synchronous /api/process body, unchanged in behavior, now run on
144         the job worker thread (DEFERRED_HARDENING_PLAN #16). Returns the same response dict
145         the sync endpoint used to return (UI compatibility); the per-video observability
146         report is still emitted in `finally:` and grafted as response["reports"].
147
148         `progress` is an optional callable(stage: str) used to surface coarse job progress.
149         Raises on unexpected internal errors ? the caller records those as a job failure
150         (after this function has already restored the pre-run segments, see below).
151         """
152         notify = progress or (lambda stage: None)
153
154         notify("hashing")
155         # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
156         (identity ? # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
157         original # req.video_path (the source URI) is kept for the DB record + report; only the file-
158         reading # consumers (hash / validators / segmenter) use the resolved local path.
159         local_video = storage.acquire_local_input(req.video_path, getattr(global_config,
160         "storage", None))
161         video_id = compute_video_id(local_video)
162         was_reingest = db_instance.get_video(video_id) is not None
163
164         # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
165         report)
166         notify("validating")
167         logger.info(f"Running validations for {req.video_path}...")
168         val_results, val_context = registry.run_all_with_context(local_video, global_config)
169         failures = [r for r in val_results if not r.passed]
170
171         # typed AppConfig: attribute access for the default segmenter (with safe fallback)
172         default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
173         "motion")
174         seg_name = req.segmenter or default_seg
175         segmenter_actual = None
176         segmentation_ran = False
177         response: Optional[Dict[str, Any]] = None
178         try:
179             if failures:
180                 # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
181                 # status; it no longer destroys the video's prior good segments.
182                 db_instance.add_video(video_id, req.video_path, "failed", [r.model_dump()
183                 for r in val_results])
184                 response = {
185                     "video_id": video_id,
186                     "status": "failed",
187                     "message": "Video failed validation checks.",
188                     "results": [r.model_dump() for r in val_results],
189                 }
190                 return response
191
192             # 2. Run segmenter & indexer
193             logger.info("[API] Video passed checks. Segmenting and indexing...")
194             db_instance.add_video(video_id, req.video_path, "processed", [r.model_dump() for
195             r in val_results])
196
197             segmenter_cls = segmenter_registry.get(seg_name)
```



```

193         if not segmenter_cls:
194             # Submit-time validation already 400s unknown names; this is the defensive
195             # in-worker path (e.g. config default pointing at an unregistered
segmenter).
196             available = list(segmenter_registry.list_available().keys())
197             response = {
198                 "video_id": video_id,
199                 "status": "failed",
200                 "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
201                 "results": [r.model_dump() for r in val_results],
202                 "play_segments": [],
203             }
204             return response
205
206         logger.info(f"[API] Using segmenter: {seg_name}")
207         notify(f"segmenting ({seg_name})")
208         segmenter_actual = seg_name
209         # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
210         # if the run yields segments; otherwise drop the new partial rows and keep the
old.
211         play_wm, cand_wm = db_instance.segment_watermarks(video_id)
212         segmenter = segmenter_cls(db_instance, global_config)
213         try:
214             result = segmenter.process_video(video_id, local_video, req.player_pool,
req.max_frames)
215         except Exception:
216             # A raising segmenter must not leave half-written rows: restore the pre-run
217             # state (same destroy-after-confirm contract as the Failure branch), then
let
218             # the job worker record the failure.
219             db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
220             raise
221         segmentation_ran = True
222
223         if isinstance(result, Failure):
224             logger.error(f"[API] Segmenter failed: {result.message}")
225             db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
226             response = {
227                 "video_id": video_id,
228                 "status": "failed",
229                 "message": f"Segmenter '{seg_name}' failed: {result.message}",
230                 "results": [r.model_dump() for r in val_results],
231                 "play_segments": [],
232             }
233             return response
234
235         segments = result.value if hasattr(result, "value") else result
236         notify("finalizing")
237         _finalize_reingest(video_id, segments, play_wm, cand_wm)
238
239         response = {

```

22. assistant (2026-06-21T09:22:53.903Z)

This fully confirms the trace. The `video_id = compute_video_id(local_video)` (line 160) hashes the fetched local file, and `add_video(video_id, req.video_path, ...)` (lines 179, 190) stores the source URI but keys on the MD5. The existing `gs://` test does NOT assert that `job['video_id']` equals `compute_video_id(fetched_bytes)`, nor that `query_play_segments(video_id)` is non-empty for the `gs://` case ? those are the gaps the new B-P1 test must close.

Let me verify the response dict includes `play_segments` and check the subprocess entrypoint test for the #281 pattern reference.

23. user (2026-06-21T09:22:55.327Z)

```
239         response = {
240             "video_id": video_id,
241             "status": "success",
242             "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",
243             "results": [r.model_dump() for r in val_results],
244             "play_segments": segments,
245         }
246         return response
247     finally:
248         # Always emit the per-video observability report (next to the video, or to
249         # report.output_dir). Never raises. Surface the paths in the response.
250         paths = emit_indexer_report(
251             db=db_instance, video_id=video_id, video_path=req.video_path,
config=global_config,
252             val_results=val_results, val_context=val_context,
253             segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
254             was_reingest=was_reingest, segmentation_ran=segmentation_ran,
255         )
256         if paths and isinstance(response, dict):
257             response["reports"] = paths
258
```

24. user (2026-06-21T09:22:56.006Z)

```
425     def test_process_local_scheme_uri_missing_still_404(env):
426         """local://<missing> still 404s ? the existence gate runs on the resolved local
key."""
427         client = env[0]
428         r = client.post("/api/process", json={"video_path": "local:///no/such/ls-
missing.mp4"})
429         assert r.status_code == 404
430
431
432     def test_process_hosted_mode_rejects_nonlocal_uri(env):
433         """Hosted mode (allowed_video_root set) must reject a bucket/Drive URI ? a tenant
can't point
434         the engine at an arbitrary bucket. Pins the security contract instead of an
incidental side effect."""
435         client, db, tmp_path, mp = env
436         root = tmp_path / "tenant"
437         root.mkdir()
438         mp.setattr(m.global_config.security, "allowed_video_root", str(root))
439         for bad in ("gs://b/x.mp4", "s3://b/x.mp4", "gdrive://Sharing/x.mp4"):
440             r = client.post("/api/process", json={"video_path": bad})
441             assert r.status_code == 400, f"{bad} -> {r.status_code}"
442         # A local file UNDER the root is still accepted (passes validation; processing
mocked elsewhere).
443         under = root / "ok.mp4"
444         under.write_bytes(b"hashable")
445         mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
446         mp.setattr(m.segmenter_registry, "get", lambda name: _FakeSeg)
447         r = client.post("/api/process", json={"video_path": str(under), "segmenter": "x"})
448         assert r.status_code == 202
449
450
451     def test_cli_nonlocal_input_fetches_and_keeps_uri(tmp_path, monkeypatch):
452         """The CLI --video-path block (a second M2 call-site) routes a bucket URI through
fetch: the
453         segmenter sees a LOCAL path and the DB keeps the source URI."""
454         db = Database(str(tmp_path / "cli.db"))
455         cfg = AppConfig()
456         cfg.output.output_dir = str(tmp_path)
```

```

457 monkeypatch.setattr(m, "load_app_config", lambda *a, **k: cfg)
458 monkeypatch.setattr(m, "Database", lambda *a, **k: db)
459 monkeypatch.setattr(m, "global_config", cfg)
460 monkeypatch.setattr(m, "db_instance", db)
461 monkeypatch.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(),
{}}))
462 monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _CaptureSeg)
463
464 def fake_fetch(uri, dst=None, *, config=None, overwrite=False):
465     with open(dst, "wb") as fh:
466         fh.write(b"cli-video-bytes")
467     return dst
468
469 monkeypatch.setattr(m.storage, "fetch", fake_fetch)
470 _CaptureSeg.last_path = None
471 monkeypatch.setattr(sys, "argv", ["prog", "--video-path",
"gs://bucket/videos/cli.mp4",
472                                "--output-dir", str(tmp_path)])
473 m.main()
474 assert _CaptureSeg.last_path is not None
475 assert not _CaptureSeg.last_path.startswith("gs://") # fetched LOCAL path
476 assert os.path.basename(_CaptureSeg.last_path) == "cli.mp4"
477 vid = m.compute_video_id(_CaptureSeg.last_path)
478 rec = db.get_video(vid)
479 assert rec is not None and rec["filepath"] == "gs://bucket/videos/cli.mp4" # source
URI kept

```

25. assistant (2026-06-21T09:23:02.371Z)

I have everything I need. Note the important nuance from
`test\_process\_hosted\_mode\_rejects\_nonlocal\_uri` (line 432): in **hosted mode**
(`allowed\_video\_root` set), `gs://` is rejected with 400. The B-P1 claim and CUJ are explicitly
scoped to **default single-user config** (allowed_video_root=None), where `gs://` is accepted. The
new test must use default config (the `env` fixture does, since `AppConfig()` has
`security.allowed_video_root=None`).

Let me confirm `_FakeSeg` writes a queryable segment and verify the `query_play_segments`
signature, plus check there's a `CompileResponse` `download_url` to confirm B-P2 scope boundaries.
Let me also verify the M2 changelog/doc in the engine to know what engine doc (if any) needs a
tweak.

26. user (2026-06-21T09:23:04.404Z)

```

backend\main.py:361: def query_play_segments(req: QueryRequest):
backend\infrastructure\database.py:427:     def query_play_segments(self, video_id: str, filters:
Dict[str, Any]) -> List[Dict[str, Any]]:
backend\infrastructure\database.py:514:     def set_job_video_id(self, job_id: str, video_id:
str) -> bool:
backend\utils\hashing.py:23: def compute_video_id(filepath: str) -> str:

```

27. user (2026-06-21T09:23:05.508Z)

C:\Users\avidu\Projects\kheIsutra\docs\DRY_RUN_CUJ.md:15:[Omitted long matching line]
C:\Users\avidu\Projects\kheIsutra\docs\DRY_RUN_CUJ.md:20:**Path B ? Full "point at a bucket
video IN the UI" (bigger, multi-session).** **Build the in-UI ingest: a net-new HTTP bucket-
ingest endpoint** (`gcs://` ? process ? DB `video_id``) + the SPA **ingest + progress** screens
(appliance S1/S2). The proper appliance front-half.
C:\Users\avidu\Projects\kheIsutra\docs\LOCAL_APPLIANCE_ARCHITECTURE.md:49:- **Absent (net-
new):** `GET /api/capabilities`, `GET /api/videos` (only single-row `get_video(id)`), `GET
/api/reels/{video_id}`, any source/proxy streaming endpoint, any **Dockerfile**, any per-video
cost ledger `[verified ? grep returns none]`.
C:\Users\avidu\Projects\kheIsutra\docs\LOCAL_APPLIANCE_ARCHITECTURE.md:87:| **Bucket URI**
(`s3://`, gs://`) | the worker already fetches --video-uri` ? local ? pipeline [verified
worker/___main__.py:98]. From the UI, a small ingest-to-process step (fetch the ref ?`

```
`/api/process`, or enqueue the worker) is net-new | design (storage layer ready) |  
C:\Users\avidu\Projects\khehsutra\docs\LOCAL_APPLIANCE_ARCHITECTURE.md:98:| Remote GPU worker  
via bucket** (reserved) | console CPU + `storage=s3`; separate GPU host runs `backend.worker  
infer` | bucket creds; a GPU host; a net-new orchestration endpoint | REAL (remote) |  
FastAPI loop and worker CLI are separate today; "remote GPU as a setting" is design ?  
selector shown disabled in MVP |  
C:\Users\avidu\Projects\khehsutra\docs\PER_RALLY_SELECTION.md:56:Reuse map (existing ?  
[verified], no change for MVP):** `POST /api/query` (list) ? `POST /api/compile` (stitch) ?  
`GET /api/highlights/{video_id}/{filename}` (download); `GET /api/config` (floor) ; `GET  
/api/jobs/{id}` (carry `result.video_id`). All net-new code is the SPA in `khehsutra` `web/`  
(Vite + React + TS + Tailwind, per `web/README.md`), plus a small set of optional real-gap  
engine enablers (§4.3) ? none required for MVP.  
C:\Users\avidu\Projects\khehsutra\docs\PER_RALLY_SELECTION.md:58:4.1 Components (net-new  
SPA)
```

28. assistant (2026-06-21T09:23:11.389Z)

This is the crux. The doc's §1 line 15 says **"Bridging 'gcs://' ? a queryable video_id in the API DB' is net-new"** ? and §2 line 20 says Path B builds **"a net-new HTTP bucket-ingest endpoint (gcs:// ? process ? DB video_id)"**. Both are FALSE per the verified trace: `POST /api/process` already accepts a gcs:// URI end-to-end under default config and produces a queryable API-DB video_id. The endpoint exists; only the SPA front-half (the ingest screen) is genuinely net-new.`

I have full verification. Let me confirm the test runner and that httpx is installed so the new test would actually run, then finalize.

29. user (2026-06-21T09:23:21.476Z)

```
httpx 0.28.1  
.. [100%]  
2 passed, 21 deselected in 6.13s
```

30. assistant (2026-06-21T09:23:24.779Z)

Both existing gs:// tests pass and httpx is present, so a new test in the same file will run. I have everything verified. The verdict is clear: B-P1 already works in code; all 3 skeptics failed to refute. The PR is test + honest doc-correction.

Let me submit the structured output.

31. user (2026-06-21T09:24:19.480Z)

Output does not match required schema: /bP1PrPlan: must have required property 'testFile', /bP1PrPlan: must have required property 'testCases', /bP1PrPlan: must have required property 'docEdits', /bP1PrPlan: must have required property 'codeChanges', /bP1PrPlan: must NOT have additional properties

32. assistant (2026-06-21T09:24:22.072Z)

I need to fix the bP1PrPlan shape to match the schema (testFile, testCases, docEdits, codeChanges).

33. user (2026-06-21T09:25:25.174Z)

Structured output provided successfully

34. assistant (2026-06-21T09:25:39.245Z)

B-P1 verdict synthesized and returned. Summary of what I verified and decided:

Verdict: B-P1 already works in code. All 3 adversarial skeptics failed to refute, and I independently re-traced and ran the two existing gs:// tests green (httpx 0.28.1 present). Under default single-user config, `POST /api/process` accepts a `gs://`/`gcs://` URI end-to-end and`

produces a queryable API-DB `video\_id` ? the endpoint the doc calls "net-new" already exists. The only divergence from the local path is the network download, whose failure modes (missing SDK/ADC) are environmental and surface as loud job failures, not code gaps.

****The B-P1 PR is therefore test + honest doc-correction, no code change:****

- ****Test**** (extend `tests/test\_api\_endpoints.py` in-place): parametrized `gs://`+`gcs://` lock asserting 202 ? job done ? `video\_id == compute\_video\_id(fetched bytes)` (not the URI string) ? `filepath` keeps the source URI ? `query\_play\_segments` non-empty (the queryability guarantee the current gs:// test omits). Uses `\_FakeSeg` (persists a segment), default `env` fixture.
- ****Doc corrections****: `khehsutra/docs/DRY\_RUN\_CUJ.md` \$1 (lines 14-15 falsely say "/api/process accepts only a local path" and bridging is "net-new"), \$2 (line 20 falsely calls the endpoint net-new), \$6 box 3; plus the half-true `LOCAL\_APPLIANCE\_ARCHITECTURE.md:87` row.

****Key honesty pins:**** the claim holds ONLY in default single-user mode (hosted mode correctly 400s gs://, already tested); the hermetic seam is in-process only so a real-subprocess #281-style guard for gs:// is not achievable with this fake ? don't over-claim it.

****B-P2**** (the genuinely net-new work) is 100% SPA front-half in `khehsutra/web` (ingest input + `useIngestJob` poll hook + `App.tsx` videoId state handoff + en/hi/kn i18n), zero engine code.

****Droplet (option 3)**** is an independent deploy task gated on provisioning + GCS creds + the merged #281 fix; not needed for B-P1.

Relevant files: `C:/Users/avidu/Projects/badminton-highlight-indexer/backend/main.py` (lines 159-190, 269-300), `C:/Users/avidu/Projects/badminton-highlight-indexer/tests/test\_api\_endpoints.py` (lines 375-398, 432-448),
`C:/Users/avidu/Projects/khehsutra/docs/DRY\_RUN\_CUJ.md`,
`C:/Users/avidu/Projects/khehsutra/docs/LOCAL\_APPLIANCE\_ARCHITECTURE.md`.